Recod.ai/LUC - Scientific Image Forgery Detection

Max Bakke Seymour[1], Einar Martin Søvik Bernsen[1], and Emre Balci[1]

[1]University of Bergen (UiB)

Abstract

TODO - "Short summary of the paper, including results. Often written last". We additionally implemented a SegNeXt-inspired segmentation baseline and compared it with the project's DINO-based baseline. While the DINO-based model achieved the best validation F1 score of 0.1850, the best pretrained SegNeXt-family variant reached a comparable score of 0.1827, showing that learned segmentation approaches can be competitive on this task.

1 Introduction

1.1 Copy-Move Forgery Detection

Copy-Move Forgery is a class of image forgery where regions of an image are duplicated and moved to other areas of the image. Commonly, techniques such as rotation, flipping and scaling of the copied region are applied, as well as post-processing such as edge smoothing, brightness adjustment and adding extra noise. These techniques can lead to forgeries which are hard to detect both through manual review and automated detectors. This method of forgery can be used within scientific images in order to fabricate results, where the complexity of the figures creates additional difficulties. [1]

1.2 Related works

Copy-move image forgery detection has been studied through the lens of many different techniques. Early approaches focused on hand-crafted descriptors extracted either densely over overlapping blocks or sparsely at keypoints, followed by nearest-neighbour matching and geometric consistency checks [2, 3]. These methods established the basic copy-move detection pipeline and showed that explicit correspondence search can be effective, but they are often sensitive to strong post-processing, complex geometric transformations, and repeated structures in the scene [3].

In more recent work, the focus has shifted towards deep learning for copy-move detection. Some methods retain the classical matching intuition while replacing hand-crafted features with learned representations and stronger dense matching modules [4]. Others formulate the task as end-to-end forgery localization, using CNN-based architectures to jointly detect duplicated content and localize manipulated regions [5]. Attention-based models have further improved localization by explicitly modelling patch relationships and co-occurrence cues [6], while more recent transformer-based approaches have explored global context modelling for copy-move detection [7]. In parallel, source-target disambiguation has also emerged as an important subproblem, since detecting near-duplicate regions alone is not always sufficient to identify the truly manipulated area [8].

One of the other architectures we tried for this task even combines both perspectives by using CNN crafted features in combination with features computed through Zernike kernels. [9]

1.3 Objectives

We aim to create a full prediction pipeline for CMFD which could perform well on the Recod.AI Scientific Image Forgery Detection dataset. Ideally, our model should be robust towards post-processing techniques. The main goal is to perform well on the competition metric, which is a modified version of F1 which computes pixelwise F1 on pairs of (*predicted mask, ground truth mask*) using the following formula [10]:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

This calculation is applied to every connected component of forged pixels in the image (belonging to either the source or target). Predictions given by the model are matched up with the best corresponding ground-truth areas using the Hungarian algorithm. For any additional predicted areas predicted by the model above the actual amount of ground-truth components, a penalty is computed:

$$P = \frac{n_{gt}}{\max(n_{pred}, n_{gt})}$$

Finally, the per-image score is given as:

$$oF1 = \text{mean}(\text{best matched pairwise F1 scores}) \times P$$

The competition metric is challenging for a few reasons. Firstly, it requires high pixel-level precision

for the output masks. In addition, a score of 0 for a given image is given if any pixels are predicted as forged on a ground-truth authentic image. Finally, it is highly punishing if any additional components are predicted, even if they should be small, as this affects the penalty P.

1.4 Contributions

This paper presents a complete main pipeline for CMFD and several alternate methods, discussing the strengths and weaknesses of the different approaches. Through a discussion and presentation of these methods, we aim to identify what directions could be promising for future work and further nuance the task of CMFD.

2 Methods

Our initial implementation was inspired by a notebook by Kaggle user Gaurav Parkhedkar [11], who successfully employed a range of different techniques to perform a score of 0.332 on the competition metric on the test set. These techniques include feature extraction through the DINOv2 model [12], sliding window inference and prediction mask post-processing. For our own implementation we build on these techniques to try other possibilities.

2.1 Dataset

We utilize the dataset supplied in the Kaggle competition [1]. Unfortunately, we were unable to test our implementation against the Kaggle test-set, as we were too late to enter the competition. As we did not have access to the test data, we instead chose to use the available training data in a train (80%), evaluate (10%), test (10%) split. The training data consists of a base 2751 forged and 2377 authentic images as well as an additional 48 forged images. As the images are too large to process directly with the model, they are each resized to size $3 \times 448 \times 448$.

2.2 Feature Extraction

To perform feature extraction on the images, we employ Meta AI's powerful DINOv2 model using the pretrained *ViT-B/14* segmentation head [12]. This model first extracts strong general-purpose features using the DINOv2 ViT backbone, then uses these features to create pixel-level predictions using the segmentation head. This model is kept frozen and applied on 14×14 size patches of the image at a time to generate features with an embedding dimension of 768. When predicting, the model processes image patches independently and reconstructs the final prediction by aggregating patch-level outputs.

2.3 Decoder

In order to create the predicted pixel mask, we employ a small decoder model on the generated DINOv2 feature map. The decoder we utilize contains three blocks and an output layer, before outputting the generated mask interpolated to output size using bilinear interpolation.

The first two blocks of the decoder apply convolutions to bring the dimensionality from $768 \to 384 \to 192$ while also applying $ReLU$ and *Dropout* with $p = 0.1$ operations in each block. In the third block, a convolution from $192 \to 96$ is applied before a final $ReLU$ and the output layer $96 \to 1$, giving the predicted mask.

2.4 Mask post-processing

To post-process masks, we applied the following operations to an input map of forged pixel probabilities:

1. Gaussian blur: A smoothing operation implemented through `scipy.ndimage.gaussian_filter`
2. Confidence threshold filtering: Creates an initial binary mask by filtering probabilities on the threshold 0.5.
3. Closing operation: Applies dilation followed by erosion to create more coherent objects, implemented using `scipy.ndimage.binary_closing` with a 5×5 structuring element.
4. Opening operation: Applies erosion followed by dilation to remove small isolated blobs, implemented using `scipy.ndimage.binary_opening` with a 3×3 structuring element.
5. Finally, remaining holes in binary objects are filled using `scipy.ndimage.binary_fill_holes`.

2.5 Sliding window inference

In order to learn global relationships between pixels in an image, it is necessary to run inference on images as a whole. This is performed on images of the same resized size as the training image input size. The generated pixel masks are upscaled through bilinear interpolation, similarly to the training image samples.

The issue with this kind of inference is that although it is able to learn the global relationships between objects in the image, it is challenging to achieve perfect pixel-level precision on images when

Model	Pretr.	Train	Ep.	Val F1
DINO baseline	Yes	900	8	**0.1850**
SegNeXt-inspired	No	500	5	0.0838
SegNeXt + pretr. enc.	Yes	900	12	0.1827

Table 1. Comparison between the DINO baseline and SegNeXt-inspired variants.

the resolution is decreased significantly. In order to give our model more pixel-level precision and allow for inference on higher-resolution images directly, we employ sliding window inference. Sliding window inference is performed by performing individual predictions on overlapping patches of the image, which are weighted by a gaussian kernel in order to create a final output pixel map. TODO: More detailed: This is combined with the global prediction performed on the whole downscaled image at once.

2.6 SegNeXt-inspired Baseline and Experimental Comparison

In addition to the main DINO-based pipeline, we implemented a learning-based baseline inspired by SegNeXt in order to compare a convolutional segmentation architecture against the project baseline. SegNeXt is motivated by the idea that semantic segmentation can benefit from a strong encoder, multi-scale feature interaction, and efficient spatial attention implemented using convolutional operations rather than transformer self-attention [13]. In particular, the original SegNeXt design uses a convolutional encoder-decoder architecture with multi-scale convolutional attention to capture contextual information efficiently. The paper reports strong performance-computation trade-offs and shows that such CNN-based designs can outperform transformer-based alternatives on several semantic segmentation benchmarks [13].

Our goal was not to reproduce the full original SegNeXt architecture exactly, but rather to implement a SegNeXt-inspired segmentation baseline within the project pipeline and evaluate whether this type of learned segmentation model could provide competitive performance on the Recod.ai/LUC scientific image forgery detection task.

We evaluated four learning rates: 10^{-4}, 5×10^{-5}, 3×10^{-5}, and 10^{-5}.

Experimental setup. The experiments were carried out on the same extracted dataset used in the rest of the project. A subset of the forged training images and corresponding masks was prepared from the Kaggle dataset. For the largest experiments, 1000 image-mask pairs were extracted, of which the effective split used during training corresponded to 900 training samples and 100 validation samples. All experiments were run with the same random seed in order to make comparisons more controlled.

Compared models. We compared three model families:

1. a DINO-based baseline used in the main project,
2. a custom SegNeXt-inspired segmentation model,
3. a pretrained backbone variant using the same lightweight decoder structure.

The DINO-based baseline served as the reference point, while the SegNeXt-inspired experiments were used to test whether a learned convolutional segmentation model could match or improve the baseline under the same training regime.

Training protocol. We trained the models with several dataset sizes and hyperparameter settings. The main experiments used training subsets of size 200, 500, and 1000, with corresponding validation subsets of size 30, 70, and 120. In practice, the largest configuration resulted in an effective split of 900 training images and 100 validation images. We evaluated learning rates of 10^{-4}, 5×10^{-5}, 3×10^{-5}, and 10^{-5}, and trained for between 3 and 12 epochs depending on the experiment.

Implementation details. All experiments were conducted with a fixed random seed of 42 and a batch size of 2. Training subset sizes of 200, 500, and 1000 were evaluated, with corresponding validation subsets of 30, 70, and 120. Learning rates of 10^{-4}, 5×10^{-5}, 3×10^{-5}, and 10^{-5} were tested across runs. This setup allowed us to compare the DINO baseline, the custom SegNeXt-inspired model, and the pretrained SegNeXt-family variant under the same overall evaluation regime.

SegNeXt-inspired implementation. The implemented model followed the general encoder-decoder idea used in modern semantic segmentation architectures.The SegNeXt paper emphasizes multi-scale convolutional feature extraction and efficient spatial attention through convolutional blocks rather than quadratic-complexity self-attention [13]. In our implementation, we adopted this design direction as a baseline for scientific forgery localization, while keeping the model lightweight enough to train within the project constraints. We additionally tested a pretrained encoder variant using a `convnext_tiny` backbone from `timm` in order to examine whether stronger pretrained visual features could improve the SegNeXt-style segmentation baseline.

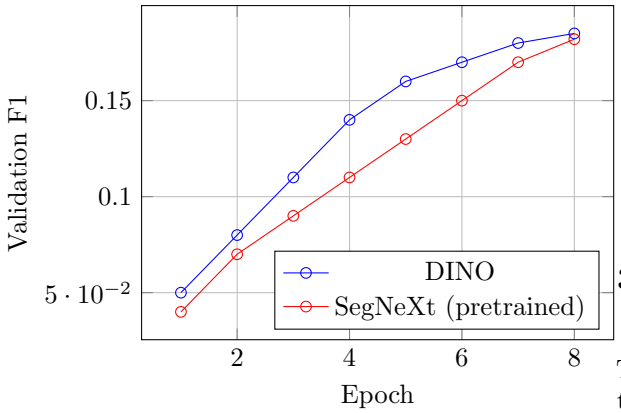

Figure 1. Training dynamics comparison between the DINO baseline and the pretrained SegNeXt-family model.

Transfer learning. Transfer learning played a central role in our experiments. The pretrained SegNeXt-family variant used a `convnext_tiny` encoder initialized with weights learned from large-scale image data, while the custom SegNeXt-inspired models were trained from scratch. This difference had a major effect on performance. The custom models remained far below the DINO baseline, whereas the pretrained SegNeXt-family model approached the DINO result closely. This indicates that, for scientific image forgery detection under limited data, representation quality is more important than architectural complexity alone.

Evaluation objective. All models were compared using the same validation metric, namely the pixel-level F1 score used in our notebook experiments. This ensured that the DINO baseline and the SegNeXt-inspired variants could be compared under the same evaluation setting. For each model family, the reported result corresponds to the run with the highest validation F1 score among the tested configurations. All compared models were evaluated under the same dataset splits and the same validation metric, ensuring a fair comparison between the DINO baseline and the SegNeXt-inspired alternatives.

3 Results

This section presents the experimental comparison between the project's DINO-based baseline and the implemented SegNeXt-inspired alternatives. We report the best validation F1 obtained for each model family under the tested training settings.

Data	DINO	S-Next (cust.)	S-Next (pretr.)
200	0.0886	0.05	0.10
500	0.14	0.07	0.15
1000	**0.1850**	0.0838	0.1827

Table 2. Validation F1 versus dataset size.

3.1 Comparison Between DINO and SegNeXt-inspired Models

Table 1 summarizes the best validation results obtained in our experiments.

The best DINO-based run achieved a validation F1 of 0.1850 with 900 training samples, a learning rate of 3×10^{-5}, and 8 epochs. The best SegNeXt-family run achieved 0.1827 under the same training size and learning rate, but required 12 epochs and a pretrained `convnext_tiny` backbone.

In contrast, the custom SegNeXt-inspired models trained without pretrained encoder weights performed substantially worse, with best validation F1 values remaining below 0.09 across all tested settings. This indicates that pretrained visual representations were important for making the SegNeXt-style approach competitive in this task.

Overall, the DINO-based baseline remained the strongest model in our experiments, but the pretrained SegNeXt-family variant achieved a very similar result. This suggests that a learned segmentation approach can be competitive, although in our experiments it did not surpass the project baseline.

Technical interpretation. These results suggest that performance on this task depends mainly on feature quality. Architectural changes alone were not sufficient. The SegNeXt-inspired model became competitive only when combined with pretrained encoder features.

Performance also improved as dataset size increased. The DINO baseline improved from 0.0886 at the smallest setting to 0.1850 at the largest setting. Without pretraining, the SegNeXt-inspired models remained weak and often peaked very early.

An additional qualitative difference between the models was training stability. The custom SegNeXt-inspired models often reached their best validation F1 very early and showed limited improvement afterwards, whereas the DINO baseline and the pretrained SegNeXt-family model improved more steadily across training.

Best configurations. The best DINO baseline achieved a validation F1 of 0.1850 using an effective training size of 900, a learning rate of 3×10^{-5}, and 8 epochs.

The best SegNeXt-family model achieved 0.1827 using the same effective training size and learn-

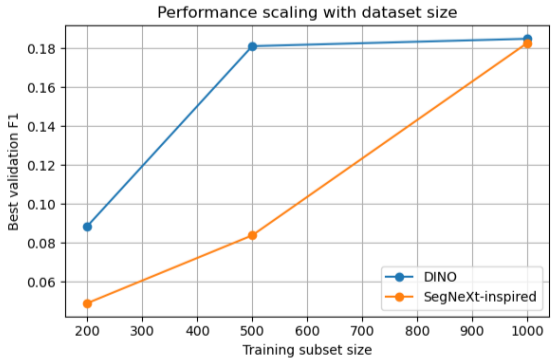

Figure 2. Validation F1 versus dataset size. The DINO baseline performs better at small dataset sizes, while the pretrained SegNeXt-family variant approaches it at the largest setting.

ing rate, but required 12 epochs and a pretrained `convnext_tiny` encoder.

TODO: Show results of experiments. NB: Without interpretation at this stage!
Align with method section.

4 Discussion

TODO: Interpret results, relate to objectives (did we achieve them?), compare against others
One issue with this task is loss function misalignment. Although cross-entropy loss correctly penalizes incorrect pixel-level predictions, it is not fully aligned with the strict conditions that the oF1 metric expects. For instance, if a small area of forged pixels is predicted on an authentic image, the impact on the loss is not large, but this makes the oF1 metric 0. This meant that we in the process of training different models observed that from epoch to epoch, even if the overall validation loss decreased, it could be the case that performance in regard to the oF1 metric became worse.
Although beyond the scope of this project, an interesting avenue for future research could be to create a differentiable loss more directly linked to the OF1 objective. As surrogates, we did employ additionally loss penalties for any forged pixels predicted on authentic images, but a more nuanced alternative would be interesting to experiment with.
Because of this mismatch, model performance is very sensitive to the operations performed in mask post-processing. We see experimenting with other post-processing techniques as a promising direction for future work. For instance, filtering out small predicted patches led to much better results, as the OF1 metric also penalizes extra component predictions on forged images.

We attribute some of the difficulty of the task to qualities of the dataset. Often, authentic images containing very similar objects, increasing the probability of a model predicting false positives. In the forged images, many of the forgeries . However, we also found examples of forgeries using small areas of the background, making the prediction task difficult.

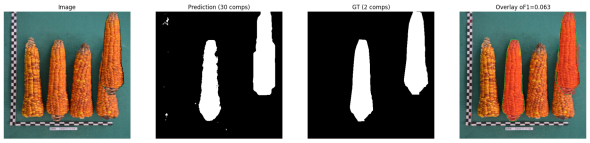

Figure 3. Results on an image using no post-processing. The model detects the copied object but also generates many small components, leading to an overall low OF1 score, even though per-pixel precision is quite strong.

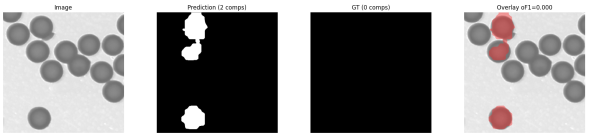

Figure 4. An authentic image containing very similar objects, causing false positives.

4.1 Deep Cross-Scale PatchMatch

The Deep Cross-Scale PatchMatch architecture was proposed by Li et al. in 2024. [9] It is an end-to-end differentiable deep-learning based architecture where one branch employs the patch match algorithm presented by Barnes et al. in 2009 [14] on different scales of each image in order to attain pixel similarities and offsets from each pixel to it's most similar pixel. The second branch runs a direct feature extraction → prediction process. Finally, the predictions of the two branches are combined through an argmax operation.
The architecture supports three-channel CMFD, but in our version it is adapted to the single-channel CMFD, using only the first part of the architecture. In addition, our implementation has some other differences from the version by Li et al. These are partly due to practical concerns, and partly due to ambiguity or lack of detail in the paper regarding specific implementation details.

Most changes were performed in the first branch, where one change performed due to practical concerns was to use a frozen pretrained CNN feature extractor. This meant that it was not necessary

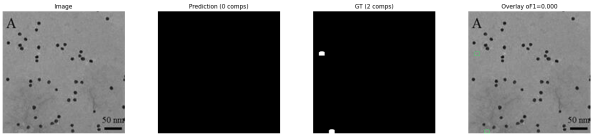

Figure 5. A forged image with the source being hard to predict due to size

to let gradients flow through the DLF and Deep Cross-Scale PatchMatch parts of the networks, saving memory and computation time. We tested using both the pretrained VGG16 and ResNet18 weights from `torchvision.models` [15, 16], achieving best performance with ResNet18. In addition, although the CNN is run multi-scale in the paper, we found that the difference in performance when using pretrained models was marginal, meaning it was more useful to save memory and time by running this model single-scale while still keeping Zernike feature extraction multi-scale. Inspired by its strong performance in our baseline, we performed feature extraction using DINOv2 in the second branch of the architecture.

In the original paper, dice losses for the final combined mask M and for the SEUnet output M_t are employed. In addition, we added two other loss terms to try and stabilize the behaviour of our algorithm. The first is a dice loss for the PatchMatch output mask M' in order to prevent this branch collapsing to predicting *authentic* for all images too soon. The second is a penalty for predictions made on authentic images in order to attempt to reduce false positives, which for each branch computes $0.5 \times \text{mean\_activation} + 0.5 * \text{max\_activation}$ and averages results together. These added losses are weighted through a tuneable hyperparameter and are empirically set to 0.8 for the M' loss and 0.25 for the empty target penalty.

The main weaknesses of this architecture are in runtime and memory consumption. The runtime is particularly dependent on the number of PatchMatch iterations which are performed. This module recurrently executes the propagation and evaluation layers "several times" [9, p. 7]. It is not clear how many iterations the authors employed, but in order to attain decent offsets, we found that a suitable amount was 24 iterations. Using this amount of iterations, PatchMatch on a single image takes 4s on an NVIDIA RTX 4070 TI Super GPU, while other components of the network take less than $0.1s$ to run.

The performance of this model was highly dependent on mask-post processing. In particular, it is crucial to perform filtering of small objects, as the architecture generated many small false positive areas on authentic images, automatically giving an OF1 score of 0 in these cases. We theorize that this tendency to predict false positives is because smaller groups of pixels are more likely to be similar to each other "by accident" than larger groups of pixels. The best version of the pipeline on validation data filters out components with a size less than 512. Although this means accepting that the model will never be able to deal with very small copy-move forgeries, this also meant that OF1 performance on the hold-out test set reached 0.538 on authentic images and 0.378 on forged images, giving a combined score of 0.452.

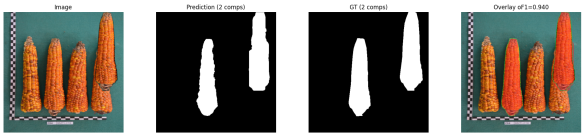

Figure 6. An example from the Kaggle dataset featuring similarities to the MC COCO-generated test set used in Li et al. [9], where our architecture achieves good performance. Note that this is the same image as in figure 1, but with post-processing applied to the predicted mask.

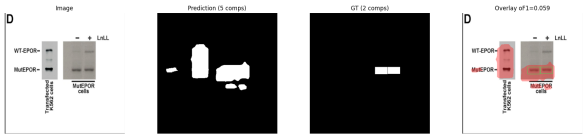

Figure 7. A scientific image where duplicated text ("MutEPOR") is predicted as a forgery and mask-post-processing combines components too aggressively.

PatchMatch's reliance on finding groups of pixels with high feature similarity may have some other fundamental issues in regards to this particular dataset. In the paper, the architecture utilized by Li et al. trains on a training set generated from MS COCO [17] by applying a random copy-move, rotation and scaling operation to a random area. It is then evaluated on a synthetic dataset generated in the same way as well as CASIA CMFD, CoMoFoD and CMH [18–20]. The common denominator for these datasets is that they feature photo-realistic images with varied objects and backgrounds. This means that there is natural variation in the qualities of pixels (for instance, they do not feature a consistent single-colour background) in contrast to many of the scientific images, which are dark, or figures containing a consistent background in which many false positives are generated from pixel-wise similarities. On the images where the copy-move is direct and not subject to post-processing, our architecture performed much better.

4.2 BusterNet

TODO: BusterNet Comments

4.3 Lessons from the SegNeXt-inspired Baseline

The SegNeXt-inspired experiments provide two main findings. First, a purely learned segmentation approach can perform competitively on this task when supported by strong pretrained visual features. The best pretrained SegNeXt-family model reached a validation F1 of 0.1827, which was close to the best DINO baseline result of 0.1850. This suggests that

the task is learnable through end-to-end segmentation and not limited to handcrafted matching pipelines.

Second, the large gap between the custom non-pretrained SegNeXt-inspired models and the pretrained variant indicates that feature quality is a major bottleneck in scientific image forgery detection. In our setting, simply adopting a segmentation architecture was not sufficient; strong initialization from pretrained encoders was crucial.

At the same time, the DINO baseline remained slightly better overall and reached its best result with fewer epochs. This suggests that the DINO-based pipeline was more stable under the available data and compute constraints. Therefore, while the SegNeXt-inspired baseline was useful as a comparison model and produced competitive performance when pretrained, it did not outperform the project baseline in our experiments.

These findings suggest that the gap between the approaches is driven not only by architecture choice, but also by representation quality and data efficiency.

4.4 Comments on Kaggle submissions

At the time of writing, the maximum oF1 score achieved on hold-out test data was 0.458, achieved by Kaggle user *orshanec*. [1] We again emphasize that our architecture cannot be directly compared with this metric due to us being unable to access the Kaggle hold-out test set.

References

[1] Kaggle. *Recod.ai/LUC - Scientific Image Forgery Detection*. https://www.kaggle.com/competitions/recodai-luc-scientific-image-forgery-detection/overview. [Online; last accessed April-2026]. 2025.

[2] J. Fridrich, D. Soukal, and J. Lukáš. "Detection of Copy-Move Forgery in Digital Images". In: (2003). URL: https://ws2.binghamton.edu/fridrich/Research/copymove.pdf.

[3] V. Christlein, C. Riess, J. Jordan, C. Riess, and E. Angelopoulou. "An Evaluation of Popular Copy-Move Forgery Detection Approaches". In: *IEEE Transactions on Information Forensics and Security* 7.6 (2012), pp. 1841–1854. DOI: 10.1109/TIFS.2012.2218597. URL: https://arxiv.org/abs/1208.3665.

[4] Y. Liu, C. Xia, X. Zhu, and S. Xu. "Two-Stage Copy-Move Forgery Detection With Self Deep Matching and Proposal SuperGlue". In: *IEEE Transactions on Image Processing* 31 (2022), pp. 541–555. DOI: 10.1109/TIP.2021.3132828. URL: https://doi.org/10.1109/TIP.2021.3132828.

[5] Y. Wu, W. Abd-Almageed, and P. Natarajan. "BusterNet: Detecting Copy-Move Image Forgery with Source/Target Localization". In: *Computer Vision – ECCV 2018* (2018), pp. 170–186. URL: https://openaccess.thecvf.com/content_ECCV_2018/papers/Rex_Yue_Wu_BusterNet_Detecting_Copy-Move_ECCV_2018_paper.pdf.

[6] A. Islam, C. Long, A. Basharat, and A. Hoogs. "DOA-GAN: Dual-Order Attentive Generative Adversarial Network for Image Copy-Move Forgery Detection and Localization". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), pp. 4676–4685. DOI: 10.1109/CVPR42600.2020.00473. URL: https://doi.org/10.1109/CVPR42600.2020.00473.

[7] Y. Liu, C. Xia, S. Xiao, Q. Guan, W. Dong, Y. Zhang, and N. Yu. "CMFDFormer: Transformer-based Copy-Move Forgery Detection with Continual Learning". In: *CoRR* abs/2311.13263 (2023). DOI: 10.48550/ARXIV.2311.13263. URL: https://arxiv.org/abs/2311.13263.

[8] M. Barni, Q.-T. Phan, and B. Tondi. "Copy Move Source-Target Disambiguation Through Multi-Branch CNNs". In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 1825–1840. DOI: 10.1109/TIFS.2020.3045903. URL: https://doi.org/10.1109/TIFS.2020.3045903.

[9] Y. Li, Y. He, C. Chen, L. Dong, B. Li, and J. Zhou. "Image Copy-Move Forgery Detection via Deep PatchMatch and Pairwise Ranking Learning". In: (2024). Also available at: https://arxiv.org/pdf/2404.17310. URL: https://ieeexplore.ieee.org/document/10736426.

[10] Kaggle. *RecodAI F1*. https://www.kaggle.com/code/metric/recodai-f1/. [Online; accessed January-2026]. 2025.

[11] G. Parkhedkar. *DINOv2 Base [0.332] — High-Res 4500px — Robust Inf.* https://www.kaggle.com/code/gauravparkhedkar/dinov2-base-0-332-high-res-4500px-robust-inf. [Online; accessed January-2026]. 2026.

[12] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. "DINOv2: Learning Robust Visual Features without Supervision". In: *arXiv preprint arXiv:2304.07193* (2023). implementation: https://github.com/facebookresearch/dinov2. URL: https://arxiv.org/abs/2304.07193.

[13] M.-H. Guo, C.-Z. Lu, Q. Hou, Z.-N. Liu, M.-M. Cheng, and S.-M. Hu. "SegNeXt: Rethinking Convolutional Attention Design for Semantic Segmentation". In: *arXiv preprint arXiv:2209.08575* (2022).

[14] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman. "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing". In: (2009). URL: https://dl.acm.org/doi/10.1145/1531326.1531330.

[15] K. Simonyan and A. Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *arXiv preprint arXiv:1409.1556* (2014). URL: https://arxiv.org/abs/1409.1556.

[16] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition". In: *arXiv preprint arXiv:1512.03385* (2016). URL: https://arxiv.org/abs/1512.03385.

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. "Microsoft COCO: Common Objects in Context". In: *European Conference on Computer Vision (ECCV)*. 2014, pp. 740–755.

[18] J. Dong, W. Wang, and T. Tan. "CASIA Image Tampering Detection Evaluation Database". In: *IEEE China Summit and International Conference on Signal and Information Processing*. 2013, pp. 422–426.

[19] D. Tralic, I. Zupancic, S. Grgic, and M. Grgic. "CoMoFoD — New Database for Copy-Move Forgery Detection". In: *Proceedings of the International Symposium ELMAR*. 2013, pp. 49–54.

[20] E. Silva, T. Carvalho, A. Ferreira, and A. Rocha. "Going Deeper into Copy-Move Forgery Detection: Exploring Image Telltales via Multi-Scale Analysis and Voting Processes". In: *Journal of Visual Communication and Image Representation* 29 (2015), pp. 16–32.

A Appendix Example